

LABORATION

Datorkonstruktion D

Mikroprogrammering

Version 4.1
2024 (AN)

1 Inledning

Syftet med laborationen är att skapa en känsla för vad som händer i en enkel dator då en maskinkodsinstruktion (även kallad assemblerinstruktion) exekveras. Syftet är också att exemplifiera några av de byggblock som en processor kan vara uppbyggd av. Efter genomförd laboration ska du ha en grundförståelse för hur en enkel dator fungerar och hur mikrokod och maskinkod tillsammans kan användas för att skapa en (relativt) hårdvaruoberoende programmeringsmodell för en assemblerprogrammerare.

Den dator som används i laborationen är helt mikroprogrammerbar och finns beskriven i häftet "Beskrivning av MIA-systemet". I det här laborationerna kommer du att få skriva all mikrokod som behövs för att kunna exekvera ett antal maskinkodsinstruktioner. Du kommer även att få skriva några maskinspråksprogram för att testa din konstruktion.

2 Praktiska råd

Precis som i all annan mjukvaruutveckling är det bra om du kan utveckla ditt program i små delar och testa dessa delar var för sig. Det är givetvis också lämpligt att spara undan gamla versioner så att du kan gå tillbaka till dessa om du av misstag skulle radera dina fungerande program.

Tänk också på att laborationen är ett examinationsmoment, det är alltså viktigt att båda personerna i en laborationsgrupp har förstått alla moment i laborationen.

3 Laborationen

Innan första labtillfället ska detta lab-PM vara noggrannt genomläst. Likaså bör de kapitel i eventuell kurslitteratur som berör mikroprogrammering vara genomlästa. Till det första lab-tillfället ska Uppgift 1 till Uppgift 3 vara förberedda, det vill säga samtliga maskinspråksprogram och mikroprogram ska ha skrivits hemma.

Till hjälp vid mikroprogrammeringen finns speciella blanketter sist i detta PM. På dessa skriver man styrordets utseende i binär form och översätter detta sedan till hexadecimal form, eftersom programmeringen av datorn sker hexadecimalt.

Uppgift 1

Skriv mikroprogrammet för följande maskininstruktioner:

Instruktion	Betydelse	Adresserings- moder	Påverkar flaggor
LOAD GR _x ,M,ADR	GR _x := PM(A)	00,01,10,11	-
STORE GR _x ,M,ADR	PM(A) := GR _x	00,10,11	-
ADD GR _x ,M,ADR	GR _x := GR _x +PM(A)	00,01,10,11	Z,N,O,C
SUB GR _x ,M,ADR	GR _x := GR _x -PM(A)	00,01,10,11	Z,N,O,C
AND GR _x ,M,ADR	GR _x := GR _x and PM(A)	00,01,10,11	Z,N
LSR GR _x ,M,Y	GR _x skiftas logiskt höger Y steg	- (ange 00)	Z,N,C utskiftad bit
BRA ADR	PC := PC+1+ADR	- (ange 00)	-
BNE ADR	PC := PC+1+ADR om Z=0, annars PC:= PC+1	- (ange 00)	-
HALT	avbryt exekv.	- (ange 00)	-

Kommentar: PC+1"i hoppinstruktionerna avser den uppräkning som görs av PC i hämtfasen.

Hämtfas

Börja med att skriva mikroprogrammet för instruktionshämtningen. Det ska börja i mikroadress 00 eftersom exekveringen av en maskininstruktions mikroprogram alltid avslutas med att mikroprogramräknaren nollställs. Hämrutinen ska göra följande:

uM adress	utför
00	ASR:= PC
01	IR:= PM, PC:= PC+1

Effektivadressberäkning

Efter instruktionshämtningen ska instruktionens effektivadress beräknas och läggas i ASR. Detta görs för alla instruktioner trots att det inte behövs för t ex BRA, som alltid är relativadresserande, eller LSR som inte berör minnet. Det behövs en mikroprogramsekvens för var och en av de fyra olika adresseringssätten. Vilken sekvens som ska utföras bestäms av M-fältet via nätet K2.

Man får alltså:

```

uM adress      utför
02              uPC:= K2(M-fältet)

```

För de fyra adresseringslägen som finns i datorn behövs följande mikrokod:

```

uM adress      utför
03              ASR:=IR, uPC:= K1(OP-fältet) ; Direktadressering

04              ASR:=PC, PC:= PC+1, uPC:= K1(OP-fältet) ; Immediate

05              ASR:= IR                      ; Indirekt adressering
06              ASR:= PM, uPC:= K1(OP-fältet)
                ; Se kommentar i Uppgift 3 om indirekt adressering!

07              AR:= IR                      ; Indexerad adressering
08              AR:= GR3+AR
09              ASR:= AR, uPC:= K1(OP-fältet)

```

Observera att K2 också måste programmeras på följande sätt:

```

Adress 0  anger startadress för direktadresseringsprogrammet
Adress 1  dito för omedelbar operand
Adress 2  dito för indirektadressering
Adress 3  dito för indexerad adressering

```

Minnesinnehållet ges av de olika sekvensernas adresser enligt ovan. Exekveringssekvenserna för instruktionerna kan läggas på godtycklig plats i mikrominnet då hoppadressen till respektive sekvens anges i K1. Det är dock lämpligt att placera dessa sekvenser omedelbart efter adressberäkningen.

Tips: M-fältet i instruktionen kommer alltid att vara satt till 11 när indexerad adressering används. För att få fram GR3 kan du alltså använda S-biten.

Förberedelseuppgift: Det finns två olika additionsinstruktioner i ALU:n. Vilken av dessa ska användas på adress 8 i mikrominnet för att utföra $AR := GR3 + AR$ och varför?

Uppgift 2

På adress \$FE i minnet finns fyra stycken fyra bitar breda tal lagrade enligt nedan:

A	B	C	D
---	---	---	---

Skriv ett assemblerprogram som använder de instruktioner du implementerade i uppgift 1 som räknar ut värdet $A + B + C + D$ och lagrar detta på position \$FF i minnet.

Exempel: Om värdet \$53AF finns på adress \$FE så ska \$0021 skrivas in på adress \$FF.

Uppgift 3

Skriv ett program som sorterar en lista med tvåkomplementstal. Listan börjar på adress \$E0 och slutar på adress \$FF. När programmet körts klart ska listan vara sorterad och det minsta värdet ska finnas på adress \$E0. Tabell 1 visar ett exempel på hur ditt program ska bete sig.

Du får implementera sorteringen på valfritt sätt, men det lättaste sättet att göra detta är antagligen genom att använda *bubble sort* enligt följande algoritm¹

1. För varje element i listan:
 - Jämför detta element med nästa element.
 - Om nästa element är mindre än det nuvarande elementet så byter du plats på dessa (och får då en sortering i stigande ordning från mest negativ till mest positiv).
2. Gå igenom listan om och om igen tills du inte längre behöver byta plats på några element. Då vet du att listan är sorterad.

Tips: Det blir antagligen lättare att skriva ditt sorteringsprogram om du implementerar en eller flera av följande instruktioner:

- CMP (samma som SUB förutom att GRx inte uppdateras)
- BGE (hoppa om GRx var större än eller lika med PM(A) i CMP-instruktionen) (båda talen använder tvåkomplement!)
- BEQ (hoppa om Z-flaggan är satt)

Du får dock, om du vill, implementera helt andra instruktioner om det passar ditt sorteringsprogram bättre. (Du kan exempelvis tänkas ha nytta av stöd för en stack genom PUSH/POP eller JSR/RTS.) Du får även, om du vill, byta ut adresseringsläget indirekt adressering mot något annat om du tycker att det skulle passa din algoritm bättre! Se även Appendix 1 för ett lämpligt flödesschema.

Tävling

Det finns inga prestandakrav i denna laboration, men för att uppmuntra er till att försöka skriva snabba program så utlyser vi en liten tävling där det gäller att skriva den snabbaste sorteringsalgoritmen. Om ni vill vara med i denna tävling så ska ni maila in er lösning (som ni sparar ifrån LMIA) till examinator. Se kurshemsidan för information om deadlines för tävlingen. Skriv Tävling och mikrokod i ärenderaden.

Notering: Vi kommer att använda fem slumpmässigt genererade listor² för att testa prestandan på era lösningar. Ni tjänar alltså inte på att optimera ert program för just den lista som anges i Tabell 1.

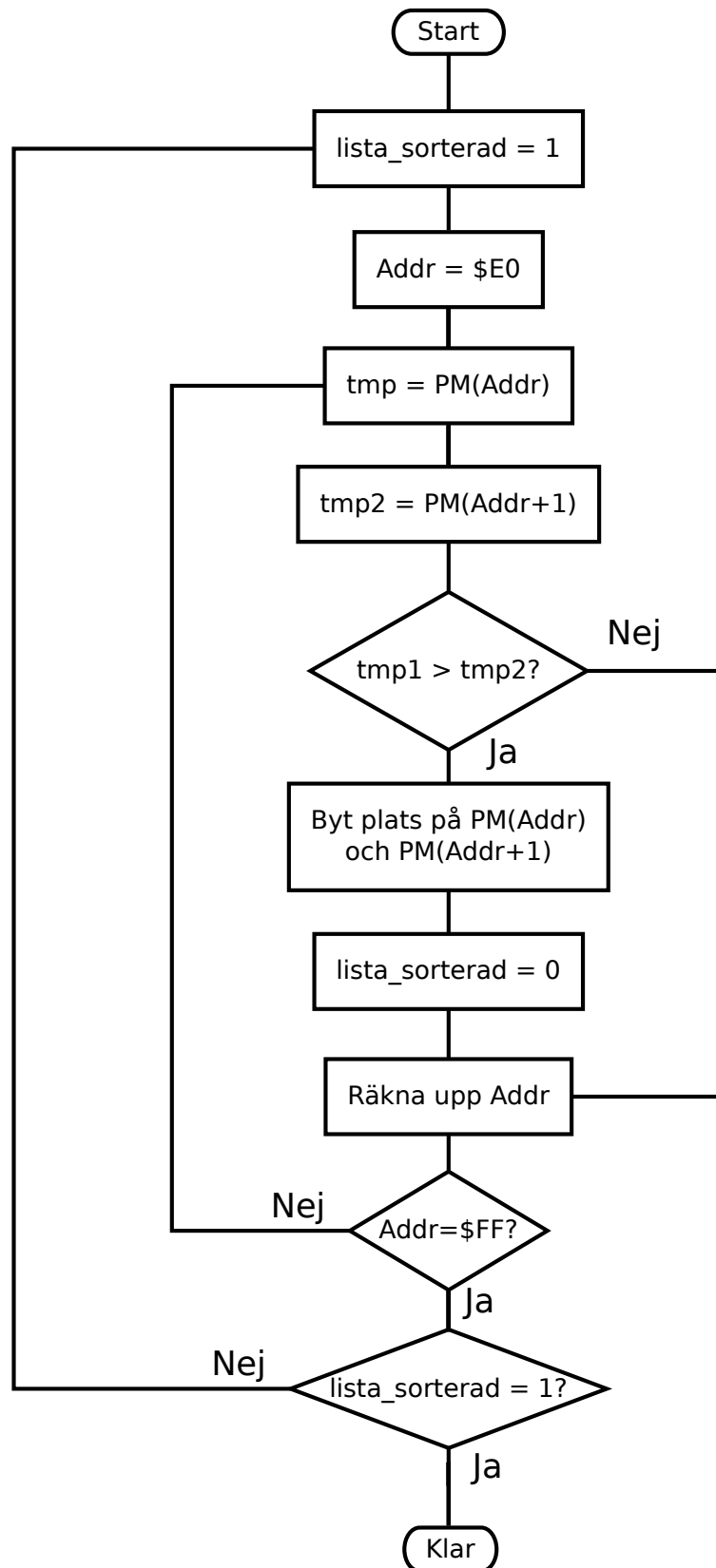
¹Det är värt att notera att Bubble-sort är en tämligen usel sorteringsalgoritm. Det finns dock några ganska enkla sätt att snabba upp den, men för de som är ute efter att skriva en riktigt snabb sorteringsalgoritm så rekommenderar jag er att antingen läsa i en lärobok om datastrukturer och algoritmer, alternativt att ta en titt på Wikipedia-sidan om "*Sorting algorithm*". (Däremot så är det inte säkert att de mest avancerade algoritmer går att anpassa till LMIA så att de är bättre än enklare algoritmer i de fall då enbart 32 element ska sorteras.)

²För de som är intresserade så kommer vi att använda 5·32·16 bitar tagna ifrån `/dev/random`.

Adress	Före	Efter
E0	92f1	8034
E1	8034	835f
E2	971b	8832
E3	99fb	90e8
E4	7ef1	92f1
E5	90e8	959f
E6	5ee7	969c
E7	3de3	971b
E8	7351	99fb
E9	53ed	9f74
EA	56a2	9fc4
EB	dea5	b11c
EC	6c5a	bd89
ED	835f	bfb0
EE	7c67	dea5
EF	ec86	ec86
F0	bd89	3de3
F1	969c	4c67
F2	5f63	53ed
F3	72d7	56a2
F4	959f	5ee7
F5	6081	5f63
F6	4c67	6081
F7	7e12	623d
F8	9fc4	6c5a
F9	b11c	7044
FA	623d	72d7
FB	8832	7351
FC	78ea	78ea
FD	9f74	7c67
FE	7044	7e12
FF	bfb0	7ef1

Tabell 1: Ett sorteringsexempel

Appendix 1: Flödesschema för bubblesort



4 Versionslog

2017 version 4.0	Tidigare version ...
2024 version 4.1	Omformulering av Uppgift 3, punkt 1 Mikroblad utgår ur dokumentet, dessa finns separat